WAKARA · DOSSIER OFFICIEL DE LIVRAISON

Dossier technique

Architecture, base de données, API, sécurité et organisation du code

Client	Mairie de Ratoma — Centre d'état civil et de l'identification (CECI Ratoma)
Prestataire	Elhadj Alhassana Camara
Document	02 / 07 — Développeurs et administrateurs techniques
Version	1.0 — 9 novembre 2026
Application	https://wakara-rtm.vercel.app

Diffusion restreinte — mairie de Ratoma et prestataire

Sommaire

1. Présentation générale
    1.1 Contexte et objectifs
    1.2 Principes de conception
2. Architecture
    2.1 Déroulement d'une requête
    2.2 Organisation du code
3. Technologies
4. Architecture fonctionnelle
5. Gestion des utilisateurs
    5.1 Authentification
    5.2 Rôles et permissions
    5.3 Protections contre l'élévation de privilèges
6. Workflow métier
    6.1 Qui peut modifier quoi, et quand
    6.2 Règles associées
7. Base de données
    7.1 Conventions
    7.2 Migrations
8. API
    8.1 Conventions
    8.2 Authentification et profil
    8.3 Utilisateurs, rôles et permissions
    8.4 Établissements
    8.5 Déclarations
    8.6 Actes et délivrances
    8.7 Pièces, audit, statistiques et paramètres
9. Sécurité
    9.1 Limites connues
10. Hébergement
    10.1 Situation actuelle
    10.2 Limites de l'offre gratuite
11. Maintenance
    11.1 Mises à jour
    11.2 Corrections et évolutions possibles

11.3 Exploitation courante

12. Annexes

12.1 Annexe A — Dictionnaire des données

12.2 Annexe B — Commandes de gestion

12.3 Annexe C — Glossaire

12.4 Annexe D — Références

1. Présentation générale

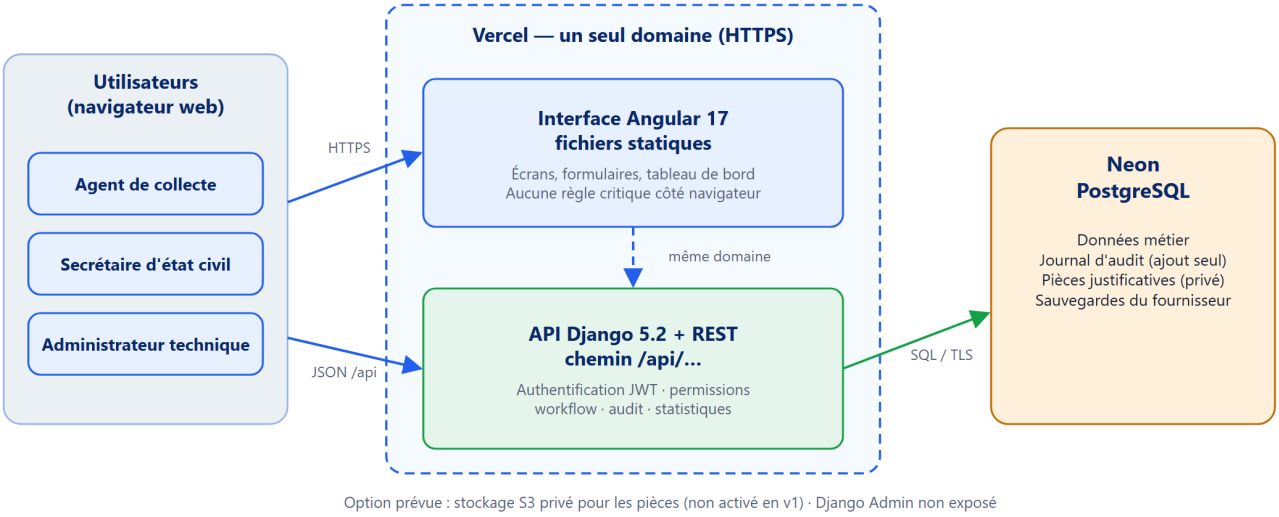
1.1 Contexte et objectifs

Wakara est la plateforme web de la Mairie de Ratoma pour gérer les déclarations de naissance reçues des établissements de santé et les actes de naissance qui en découlent. Elle couvre tout le cycle : saisie, transmission, complément, validation, numérotation, impression, délivrance, statistiques et traçabilité. Le périmètre fonctionnel est décrit dans le cahier des charges (document 01).

1.2 Principes de conception

- **Le serveur décide.** L'interface n'applique aucune règle critique : elle affiche ce que l'API autorise (`available_transitions`, `can_edit`, `permissions`) et l'API refuse tout le reste.
- **Refus par défaut.** Une action qui n'est pas explicitement déclarée dans la table des permissions d'une vue est refusée.
- **Aucune suppression physique.** Les éléments sont archivés ou désactivés ; le journal d'audit et l'historique des statuts sont en ajout seul.
- **Tout est tracé.** Auteur, date, adresse IP, ancienne et nouvelle valeur de chaque modification.
- **Aucune règle administrative inventée.** Les hypothèses de travail sont configurables et listées pour validation.

2. Architecture



Architecture générale : interface, API et base de données

L'interface Angular est servie comme un ensemble de fichiers statiques. Elle dialogue avec l'API Django en JSON, sur le même domaine (chemin `/api/…`), ce qui évite toute configuration d'échange entre domaines. L'API lit et écrit dans une base PostgreSQL.

2.1 Déroulement d'une requête

1. Le navigateur envoie la requête avec le jeton d'accès dans l'en-tête `Authorization: Bearer …`.
2. L'intercepteur de contexte d'audit retient l'adresse IP (selon `NUM_PROXIES`) et le navigateur.
3. La classe de permission vérifie le jeton, l'état du compte (actif, mot de passe à changer ?) et le droit requis par l'action.
4. La vue délègue à un **service métier** (`declarations/services.py`, `actes/services.py`) qui applique le workflow dans une transaction avec verrous.
5. Le journal d'audit enregistre l'opération ; la réponse JSON est renvoyée.

2.2 Organisation du code

```
wakara-rtm1/
├── backend/                API Django
│   ├── wakara/             réglages, urls, wsgi
│   ├── core/               workflow, permissions, mixins, paramètres, commandes
│   ├── accounts/           utilisateurs, rôles, permissions, authentification
│   ├── hospitals/          établissements de santé
│   ├── declarations/       déclarations, enfant, parents, déclarant, historique
│   ├── actes/              actes, séquences, impressions, délivrances
│   ├── documents/          pièces justificatives (stockage privé)
│   ├── audit/              journal d'audit, historiques par dossier
│   ├── stats/              tableau de bord
│   ├── tests/              161 tests automatisés
│   └── build.py            étape de build Vercel (migrations)
├── frontend/               interface Angular 17
│   └── src/app/
│       ├── core/           API, authentification, gardes, intercepteur, modèles
│       ├── shared/         composants et directives réutilisables
│       ├── layout/         structure (menu latéral, en-tête)
│       └── features/       un composant par écran
├── docs/                   documentation et dossier de livraison
└── vercel.json             déclaration des deux services
```

Application Django	Responsabilité
`core`	Mixins (horodatage, archivage logique, ajout seul), machine à états du workflow, catalogue de permissions, vues d'audit génériques, paramètres du site, commandes de gestion
`accounts`	Utilisateur, rôles, permissions, ajustements par utilisateur, authentification et jetons
`hospitals`	Établissements (désactivation et archivage, jamais de suppression), code automatique, préremplissage de Ratoma
`declarations`	Déclaration, enfant, parents, déclarant, historique des statuts, compteur de références, exports
`actes`	Acte (références du registre, numéro officiel), séquences, impressions, délivrances, document imprimable
`documents`	Pièces justificatives en stockage privé, téléchargement authentifié et tracé
`audit`	Journal d'audit en ajout seul, contexte de requête, historiques par dossier
`stats`	Tableau de bord calculé depuis la base

3. Technologies

Composant	Version	Usage
Python	3.12 ou plus (testé avec 3.14)	Langage du backend
Django	5.2.19	Cadre web, ORM, migrations, administration
Django REST Framework	3.18.3	API REST, sérialisation, permissions

Composant	Version	Usage
djangorestframework-simplejwt	≥ 5.3, < 6.0	Jetons JWT (rotation, liste noire)
django-filter	26.2	Filtres des listes
django-cors-headers	4.9.0	Échanges entre origines (développement)
psycopg	3.3.8	Pilote PostgreSQL
Pillow	13.0.0	Contrôle des images
django-storages	1.14.6	Stockage S3 privé des pièces (option)
drf-spectacular	0.30.0	Documentation OpenAPI (développement)
python-decouple	3.8	Lecture des variables d'environnement
Angular	17.3	Interface (composants autonomes, signaux, routes à chargement différé)
TypeScript	5.3	Langage de l'interface
Chart.js	4.4	Graphiques du tableau de bord
RxJS	7.8	Flux de données asynchrones
Karma, Jasmine	6.4, 5.1	Tests unitaires de l'interface
PostgreSQL (Neon)	15 ou plus	Base de données de production (SQLite en développement)

> Les versions sont celles installées lors de la livraison. Le fichier `backend/requirements.txt` fixe des plages de versions ; `frontend/package-lock.json` fixe les versions exactes de l'interface.

4. Architecture fonctionnelle

Wakara se compose de huit modules fonctionnels. Chacun correspond à un ou plusieurs écrans de l'interface, à des points d'accès de l'API et à des tables de la base.

Module	Fonctions	Écrans (chemins)	Application
Déclarations de naissance	Saisie, brouillon, transmission, pièces, corrections, export	`/declarations`, `/declarations/new`, `/declarations/:id`, `/secretariat`	declarations
Actes de naissance	Complément, numéro officiel, aperçu, impression, délivrances	`/declarations/:id/acte`, `/actes`, `/actes/:id/document`, `/deliveries`	actes
Établissements	Création, désactivation, archivage, restauration	`/hospitals`	hospitals
Utilisateurs	Comptes, rôles attribués, mot de passe, désactivation	`/users`, `/profile`	accounts
Rôles et permissions	Permissions par rôle, ajustements individuels	`/roles`	accounts

Module	Fonctions	Écrans (chemins)	Application
Validation administrative	File de validation, décision finale, numérotation	`/validation`	declarations, actes
Statistiques	Tableau de bord, filtres, cartes cliquables	`/dashboard`	stats
Traçabilité et audit	Journal d'activité, historiques des actes, exports	`/audit` , `/history` , `/history/:id`	audit

Autres écrans : connexion (`/login`), paramètres (`/settings`), accès refusé et page introuvable.

5. Gestion des utilisateurs

5.1 Authentification

- **Connexion** par identifiant ou adresse e-mail et mot de passe. En cas d'échec, le message est identique que le compte existe ou non.
- **Jetons JWT** : jeton d'accès (60 minutes par défaut) et jeton de rafraîchissement (7 jours par défaut) avec rotation ; le jeton de rafraîchissement est mis en liste noire à la déconnexion.
- **Révocation** : les jetons d'un utilisateur sont révoqués lors d'un changement de mot de passe, d'une réinitialisation ou d'une désactivation ; un compte désactivé est refusé immédiatement, même avec un jeton déjà émis.
- **Verrouillage** : après 5 échecs, le compte est bloqué 15 minutes (réglable) ; le compteur est tenu en base, de façon atomique.
- **Mot de passe** : validateurs Django (longueur, mot de passe courant, similarité) ; changement obligatoire à la première connexion et après une réinitialisation.
- **Stockage des jetons côté navigateur** : `sessionStorage` par défaut ; « se souvenir de moi » utilise `localStorage` .

5.2 Rôles et permissions

Les droits de chaque utilisateur sont la **somme des permissions de ses rôles actifs**, plus les permissions accordées individuellement, moins celles retirées individuellement. Le drapeau « superutilisateur » de Django n'accorde **aucun** droit métier : il sert seulement à la maintenance technique.

Permission	Libellé	Admin	Secrétaire	Agent
`declaration.view`	Consulter ses déclarations	✓	✓	✓
`declaration.view_all`	Consulter toutes les déclarations	✓	✓	—
`declaration.create`	Créer une déclaration	✓	—	✓
`declaration.update`	Modifier une déclaration	✓	—	✓
`declaration.transmit`	Transmettre une déclaration	✓	—	✓
`declaration.archive`	Archiver une déclaration	✓	—	—
`acte.view`	Consulter les actes	✓	✓	—

Permission	Libellé	Admin	Secrétaire	Agent
`acte.complete`	Prendre en charge un dossier	✓	✓	—
`acte.update`	Compléter un acte	✓	✓	—
`acte.submit`	Soumettre un acte à validation	✓	✓	—
`acte.print`	Imprimer un acte	✓	✓	—
`acte.deliver`	Enregistrer une délivrance	✓	✓	—
`dossier.validate`	Valider définitivement	✓	—	—
`dossier.reject`	Rejeter ou retourner un dossier	✓	—	—
`document.view`	Consulter les pièces justificatives	✓	✓	✓
`document.upload`	Ajouter une pièce justificative	✓	✓	✓
`hospital.view`	Consulter les établissements	✓	✓	✓
`hospital.create`	Créer un établissement	✓	✓	—
`hospital.update`	Modifier un établissement	✓	✓	—
`hospital.archive`	Archiver un établissement	✓	✓	—
`user.view`	Consulter les utilisateurs	✓	—	—
`user.create`	Créer un utilisateur	✓	—	—
`user.update`	Modifier un utilisateur	✓	—	—
`user.manage_status`	Activer / désactiver un compte	✓	—	—
`user.assign_roles`	Attribuer des rôles et permissions	✓	—	—
`user.reset_password`	Réinitialiser un mot de passe	✓	—	—
`role.view`	Consulter les rôles	✓	—	—
`role.manage`	Gérer les rôles et leurs permissions	✓	—	—
`stats.view`	Consulter les statistiques	✓	✓	—
`data.export`	Exporter les données	✓	—	—
`audit.view`	Consulter les journaux d'audit	✓	—	—
`settings.manage`	Gérer les paramètres	✓	—	—

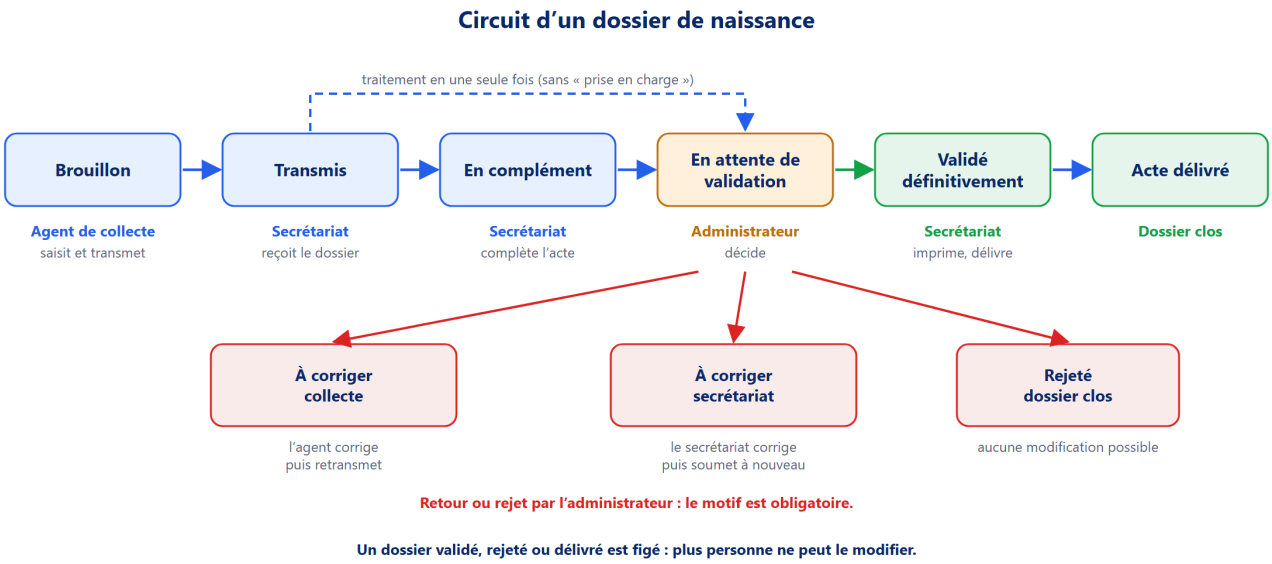
Ces rôles sont ceux créés à l'installation ; l'administrateur peut les ajuster dans l'écran « Rôles et permissions ».

5.3 Protections contre l'élévation de privilèges

- Nul ne peut modifier ses propres rôles, permissions ou statut.
- Le rôle administrateur conserve toujours `user.view`, `user.assign_roles`, `role.view` et `role.manage` (impossible de se verrouiller dehors).
- Les drapeaux privilégiés (`is_superuser`, `is_staff`, rôles) sont ignorés à la création d'un compte et sur le profil personnel.

- Utilisateurs et rôles ne se suppriment pas.

6. Workflow métier



Circuit de traitement d'un dossier

Le circuit est une **machine à états** (`core/workflow.py`) appliquée par le serveur. Chaque transition nomme la permission qui l'autorise ; un statut ne peut jamais être imposé directement par une requête.

De	Vers	Permission	Motif
Brouillon	Transmis au secrétariat	`declaration.transmit`	
Transmis au secrétariat	En complément	`acte.complete`	
Transmis au secrétariat	En attente de validation	`acte.submit`	
En complément	En attente de validation	`acte.submit`	
En attente de validation	Validé définitivement	`dossier.validate`	
En attente de validation	À corriger — collecte	`dossier.reject`	Obligatoire
En attente de validation	À corriger — secrétariat	`dossier.reject`	Obligatoire
En attente de validation	Rejeté	`dossier.reject`	Obligatoire
À corriger — collecte	Transmis au secrétariat	`declaration.transmit`	
À corriger — secrétariat	En attente de validation	`acte.submit`	
Validé définitivement	Acte délivré	`acte.deliver` (par l'enregistrement d'une délivrance)	

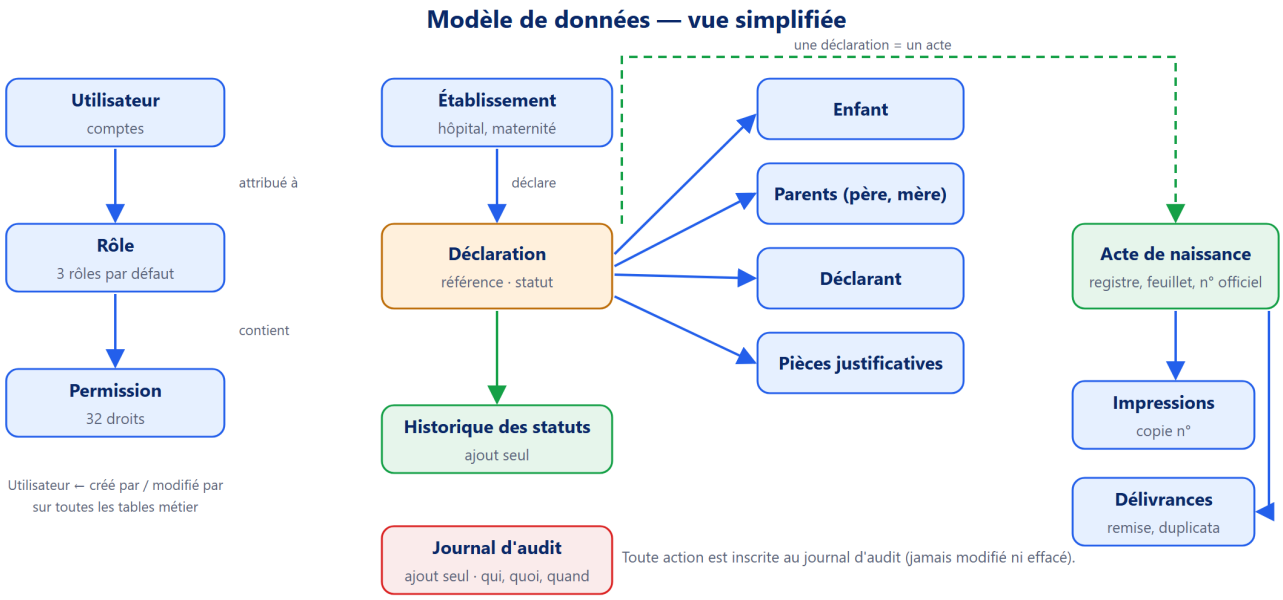
6.1 Qui peut modifier quoi, et quand

Statut du dossier	Modification possible par
Brouillon, À corriger — collecte	L'agent de collecte (ses propres dossiers)
Transmis, En complément, À corriger — secrétariat	Le secrétariat (données de la déclaration et acte)
En attente de validation	Seul le validateur (administrateur) ; le dossier reste dans sa file jusqu'à la décision
Validé, Rejeté, Acte délivré	Personne : dossier figé

6.2 Règles associées

- **Informations minimales** : transmission (date de naissance, sexe, nom de l'enfant, nom de la mère) et soumission de l'acte (date de déclaration, nom du déclarant) ; réglables par `DECLARATION_REQUIRED_FOR_TRANSMISSION` et `ACTE_REQUIRED_FOR_SUBMISSION`.
- **Numéro officiel** : saisi à la validation (par défaut), unique par année et registre, attribué dans une transaction verrouillée, jamais écrasé. Numérotation automatique disponible (`ACTE_AUTO_NUMBERING`), désactivée.
- **Validation avec champs vides** : le serveur répond 409 (`incomplete_confirmation_required`) avec la liste des champs ; rien n'est écrit tant que `accept_incomplete` n'est pas confirmé ; les champs manquants sont alors consignés dans l'historique et l'audit.
- **Référence interne** : `WKR-AAAA-NNNN`, produite par un compteur verrouillé ; distincte du numéro officiel de l'acte et de l'identifiant technique (UUID).

7. Base de données



Modèle de données — vue simplifiée

La base compte **21 tables** métier. Le dictionnaire complet (champ par champ) figure en annexe A.

7.1 Conventions

- `uuid` : identifiant technique exposé par l'API (l'identifiant numérique interne n'est jamais exposé).
- `created_at`, `updated_at` : posés par le serveur ; `created_by`, `updated_by` : posés à partir de l'utilisateur connecté, jamais depuis le corps de la requête.
- `is_archived`, `archived_at`, `archived_by` : archivage logique ; aucune ligne métier n'est supprimée.
- Le journal d'audit et l'historique des statuts refusent toute modification ou suppression (au niveau de l'application).

7.2 Migrations

Chaque application versionne ses migrations. En production, elles sont appliquées automatiquement à chaque déploiement (`backend/build.py`) ; le contrôle `python manage.py makemigrations --check` ne doit signaler aucune migration manquante.

8. API

8.1 Conventions

- Toutes les routes sont sous `/api/` ; les échanges sont en JSON, sauf les exports CSV et le téléchargement de pièces.
- Authentification : en-tête `Authorization: Bearer <jeton d'accès>` (sauf connexion et textes publics).
- Les ressources sont désignées par leur `uuid`. Les listes sont paginées et filtrables (recherche, tri, filtres propres à chaque ressource).
- Erreurs : 400 (données invalides), 401 (non connecté), 403 (droit manquant), 404, 409 (confirmation nécessaire), avec un message en français.
- Documentation interactive (Swagger) : `/api/docs/` et schéma OpenAPI `/api/schema/` , **uniquement en développement** (désactivées en production).

8.2 Authentification et profil

Méthode	Chemin	Rôle	Permission
POST	`/api/auth/login/`	Connexion ; retourne les jetons et le profil	Ouvert
POST	`/api/auth/refresh/`	Renouvelle le jeton d'accès	Ouvert
POST	`/api/auth/logout/`	Déconnexion ; met le jeton de rafraîchissement en liste noire	Ouvert
GET, PATCH	`/api/auth/me/`	Profil de l'utilisateur connecté	`Connecté`
POST	`/api/auth/change-password/`	Changement de mot de passe (retourne un nouveau couple de jetons)	`Connecté`
GET	`/api/public/settings/`	Textes de la page de connexion	Ouvert
GET	`/api/inbox/`	Compteurs de travail en attente (pastilles du menu)	`declaration.view`

8.3 Utilisateurs, rôles et permissions

Méthode	Chemin	Rôle	Permission
GET	`/api/users/`	Liste des utilisateurs	`user.view`
POST	`/api/users/`	Crée un utilisateur (mot de passe temporaire)	`user.create`
GET	`/api/users/{uuid}/`	Détail d'un utilisateur	`user.view`
PATCH	`/api/users/{uuid}/`	Modifie un utilisateur	`user.update`
POST	`/api/users/{uuid}/activate/`	Active un compte	`user.manage_status`
POST	`/api/users/{uuid}/deactivate/`	Désactive un compte	`user.manage_status`
GET, POST	`/api/users/{uuid}/roles/`	Rôles d'un utilisateur ; attribution	`user.assign_roles`
POST	`/api/users/{uuid}/roles/remove/`	Retire un rôle	`user.assign_roles`
GET, POST, DELETE	`/api/users/{uuid}/permissions/`	Ajustements individuels de permissions	`user.assign_roles`
POST	`/api/users/{uuid}/reset-password/`	Réinitialise le mot de passe (temporaire)	`user.reset_password`
GET	`/api/roles/`	Liste des rôles	`role.view`
POST, PATCH, PUT	`/api/roles/` et `/api/roles/{id}/`	Crée ou modifie un rôle	`role.manage`
PUT	`/api/roles/{id}/permissions/`	Définit les permissions d'un rôle	`role.manage`
GET	`/api/permissions/`	Catalogue des permissions	`role.view`

8.4 Établissements

Méthode	Chemin	Rôle	Permission
GET	`/api/hospitals/`	Liste (filtres : commune, actif, type ; recherche par nom ou code)	`hospital.view`
POST	`/api/hospitals/`	Crée un établissement (code automatique)	`hospital.create`
GET	`/api/hospitals/{uuid}/`	Détail	`hospital.view`
PATCH, PUT	`/api/hospitals/{uuid}/`	Modifie un établissement	`hospital.update`
POST	`/api/hospitals/{uuid}/activate/` · `deactivate/`	Active ou désactive	`hospital.update`
DELETE	`/api/hospitals/{uuid}/`	Archive (aucune suppression)	`hospital.archive`
POST	`/api/hospitals/{uuid}/restore/`	Restaure un établissement archivé	`hospital.archive`

8.5 Déclarations

Méthode	Chemin	Rôle	Permission
GET	`/api/declarations/`	Liste (recherche, filtres de statut, établissement, dates, responsable)	`declaration.view`
POST	`/api/declarations/`	Crée une déclaration (brouillon)	`declaration.create`
GET	`/api/declarations/{uuid}/`	Détail, transitions disponibles, droits d'édition	`declaration.view`
PATCH, PUT	`/api/declarations/{uuid}/`	Modifie les données de la déclaration	`declaration.update` ou `acte.update`
DELETE	`/api/declarations/{uuid}/`	Archive (brouillon ou rejeté)	`declaration.archive`
POST	`/api/declarations/{uuid}/transition/`	Change le statut (transmettre, prendre en charge, soumettre, valider, retourner, rejeter)	`Selon la transition`
GET	`/api/declarations/export/`	Export CSV de la liste filtrée (tracé)	`data.export`
GET	`/api/declarations/agents/`	Responsables présents (filtre)	`declaration.view_all`

8.6 Actes et délivrances

Méthode	Chemin	Rôle	Permission
GET	`/api/actes/`	Liste des actes validés ou délivrés	`acte.view`
GET	`/api/actes/{uuid}/`	Détail d'un acte	`acte.view`
PATCH, PUT	`/api/actes/{uuid}/`	Complète l'acte (registre, feuillet, dates, mentions)	`acte.update`
GET	`/api/actes/{uuid}/document/`	Données du document imprimable	`acte.view`
POST	`/api/actes/{uuid}/print/`	Enregistre une impression	`acte.print`
GET	`/api/actes/{uuid}/prints/`	Historique des impressions	`acte.view`
POST	`/api/actes/{uuid}/deliver/`	Enregistre une délivrance (duplicata si déjà délivré)	`acte.deliver`
GET	`/api/actes/deliveries/`	Liste des délivrances	`acte.view`
GET	`/api/actes/deliveries/summary/`	Compteurs des délivrances	`acte.view`
GET	`/api/actes/deliveries/{uuid}/`	Détail d'une délivrance	`acte.view`

8.7 Pièces, audit, statistiques et paramètres

Méthode	Chemin	Rôle	Permission
GET, POST	`/api/documents/`	Liste ; ajout d'une pièce justificative	`document.view · document.upload`
GET, DELETE	`/api/documents/{uuid}/`	Détail ; archivage d'une pièce	`document.view · document.upload`

Méthode	Chemin	Rôle	Permission
GET	`/api/documents/{uuid}/download/`	Téléchargement authentifié et tracé	`document.view`
GET	`/api/audit/`	Journal d'activité (filtres)	`audit.view`
GET	`/api/audit/{id}/ · actions/`	Détail d'un événement ; liste des actions	`audit.view`
GET	`/api/audit/export/`	Export CSV du journal (tracé)	`data.export`
GET	`/api/history/`	Historiques par dossier (recherche, filtres)	`audit.view`
GET	`/api/history/{id}/ · summary/`	Histoire d'un dossier ; indicateurs du jour	`audit.view`
GET	`/api/stats/dashboard/`	Tableau de bord (période et filtres)	`stats.view`
GET, PATCH	`/api/settings/`	Paramètres modifiables	`settings.manage`

9. Sécurité

Domaine	Mesures en place
Authentification	Jetons à durée limitée, rotation et liste noire ; révocation sur changement de mot de passe, réinitialisation et désactivation ; verrouillage de compte ; message d'erreur uniforme ; changement de mot de passe obligatoire ; limitation du débit de la connexion
Autorisations	Permissions vérifiées par le serveur à chaque requête, refus par défaut ; visibilité des dossiers limitée (l'agent ne voit que les siens) ; protections contre l'élévation de privilèges ; superutilisateur Django sans droit métier
Intégrité des données	Aucune suppression physique ; journal d'audit et historique des statuts en ajout seul ; verrous de ligne pour la numérotation et le workflow ; contrainte d'unicité du numéro par année et registre
Traçabilité	Chaque opération est consignée (auteur, date, adresse IP, navigateur, modifications, motif) ; consultation et export du journal eux-mêmes tracés ; mots de passe et jetons jamais écrits dans le journal
Pièces justificatives	Extension et signature du fichier vérifiées, taille limitée, nom aléatoire, stockage hors de toute adresse publique, `Cache-Control: no-store`, accès filtré par la visibilité du dossier, chaque téléchargement tracé
Transport et navigateur	HTTPS imposé et HSTS en production, cookies sécurisés ; documentation de l'API désactivée en production ; Django Admin non exposé sur l'hébergement actuel
Adresse IP du client	L'en-tête `X-Forwarded-For` n'est lu qu'avec le nombre de relais de confiance déclaré (`NUM_PROXIES`) ; une valeur invalide est écartée
Exports	Réservés à la permission `data.export` ; cellules CSV neutralisées contre l'injection de formules
Secrets	Aucun secret dans le code ni dans le dépôt ; tout passe par des variables d'environnement

9.1 Limites connues

- La limitation du débit utilise un cache mémoire propre à chaque instance : en environnement sans serveur, elle est moins stricte (le verrouillage de compte, lui, est fiable car tenu en base). Un cache partagé (Redis) est

recommandé.

- Un administrateur de la base de données peut contourner l'ajout seul du journal par une requête SQL directe. En production, retirer les droits de modification et de suppression à l'utilisateur applicatif : `REVOKE UPDATE, DELETE, TRUNCATE ON audit_auditlog, declarations_statushistory FROM <utilisateur_applicatif>;`
- Axes d'audit encore à approfondir : cycle de vie des jetons, limitation de débit générale, ordre d'écriture du journal, vulnérabilités des bibliothèques de l'interface (voir le rapport de tests).

10. Hébergement

10.1 Situation actuelle

Élément	Valeur
Adresse	https://wakara-rtm.vercel.app
Interface et API	Vercel, projet `wakara-rtm`, deux services sur un même domaine (voir `vercel.json`) : `backend` (Django, chemin `/api`) et `frontend` (Angular)
Base de données	Neon (PostgreSQL), base `wakara-db`, région `iad1` (États-Unis), offre gratuite
Déploiement	Automatique à chaque envoi sur la branche `main` du dépôt
Initialisation de la base	`backend/build.py` exécute `migrate` et `setup_initial_data` à chaque déploiement de **production** uniquement (les déploiements de prévisualisation ne touchent pas la base)
Pièces justificatives	Dans la base de données (privées), sauvegardées avec elle ; un bucket S3 privé reste possible
Nom de domaine	Domaine Vercel `*.vercel.app` ; un domaine personnalisé se configure dans Vercel (`ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`)

> **Décision à prendre par la mairie.** Les données d'état civil sont des données personnelles : le lieu d'hébergement (local, national ou international) doit être choisi en connaissance de cause. Le dépôt reste déployable sur un serveur classique (gunicorn et un serveur web) : voir le guide d'installation et d'exploitation (document 04).

10.2 Limites de l'offre gratuite

- Base Neon : 0,5 Go ; chaque photo de fiche pèse 1 à 3 Mo : surveiller le volume ou basculer les pièces vers un stockage S3.
- Envois limités à 4 Mo par fichier ; l'interface réduit automatiquement les photos.
- Le service « Vercel Services » est en version bêta : en cas d'échec de déploiement, consulter le journal de build.

11. Maintenance

11.1 Mises à jour

- **Dépendances** : surveiller les avis de sécurité de Django, Django REST Framework et Angular ; planifier la montée de version d'Angular 17 (des vulnérabilités sont signalées par `npm audit`).
- **Procédure de modification** : modifier le code, lancer l'ensemble des contrôles (`python manage.py test tests`, `makemigrations --check`, `npm run build`, `tsc -p tsconfig.spec.json`), puis envoyer sur `main` : Vercel déploie et applique les migrations.
- **Base de données** : toute évolution de structure passe par une migration versionnée ; ne jamais modifier la base à la main.

11.2 Corrections et évolutions possibles

- Génération d'un PDF de l'acte par le serveur, après approbation du modèle officiel.
- Procédure de rectification et mentions marginales après validation.
- Réinitialisation du mot de passe par e-mail.
- Cache partagé pour la limitation du débit, et supervision (alertes d'erreurs, journal technique centralisé).
- Reprise de données historiques depuis les registres papier.
- Autres actes d'état civil (mariage, décès).

11.3 Exploitation courante

- Vérifier chaque semaine la sauvegarde de la base et la place disponible.
- Revoir les comptes actifs et les droits au moins chaque trimestre ; désactiver les comptes inutilisés.
- Tenir un registre d'exploitation (incidents, restaurations, mises à jour) : voir le document 04.

12. Annexes

12.1 Annexe A — Dictionnaire des données

Structure exacte des tables au moment de la livraison, produite à partir des modèles. Les colonnes communes à toutes les tables métier (`uuid`, dates, auteurs, archivage) sont décrites dans les conventions du chapitre 7 et ne sont pas répétées table par table.

Utilisateur — table accounts_customuser

Compte de connexion (identifiant, coordonnées, état du compte). Clé primaire : `id`.

Champ	Type	Description	Contraintes
`password`	Texte (128)	Empreinte du mot de passe (jamais le mot de passe en clair)	
`last_login`	Date et heure	Dernière connexion	facultatif
`is_superuser`	Oui / non	Compte de maintenance Django (aucun droit métier)	

Champ	Type	Description	Contraintes
`username`	Texte (150)	Identifiant de connexion	unique
`first_name`	Texte (150)	Prénom	
`last_name`	Texte (150)	Nom	
`email`	E-mail	Adresse e-mail	
`is_staff`	Oui / non	Accès à l'administration Django (maintenance)	
`is_active`	Oui / non	Actif (oui) ou désactivé (non)	
`date_joined`	Date et heure	Date de création du compte	
`uuid`	UUID	Identifiant technique exposé par l'API	unique
`phone`	Texte (20)	Téléphone, enregistré au format +224 624 62 94 77	
`address`	Texte long	Adresse / domicile	
`hospital`	Lien → Hospital	Établissement de rattachement (facultatif)	facultatif
`must_change_password`	Oui / non	Oui : le mot de passe doit être changé avant toute action	
`failed_login_count`	Entier	Nombre d'échecs de connexion consécutifs	
`locked_until`	Date et heure	Compte verrouillé jusqu'à cette date	facultatif
`created_by`	Lien → CustomUser	Auteur de la création (posé par le serveur)	facultatif
`created_at`	Date et heure	Date de création (posée par le serveur)	
`updated_at`	Date et heure	Date de dernière modification (posée par le serveur)	

Rôle — table accounts_role

Ensemble de permissions (administrateur, secrétaire, agent de collecte, ou rôle créé par l'administrateur). Clé primaire : `id`.

Champ	Type	Description	Contraintes
`created_at`	Date et heure	Date de création (posée par le serveur)	
`updated_at`	Date et heure	Date de dernière modification (posée par le serveur)	
`codename`	Texte (50)	Code technique du rôle	unique
`name`	Texte (100)	Nom du rôle	
`description`	Texte long	Description	
`is_active`	Oui / non	Actif (oui) ou désactivé (non)	
`is_system`	Oui / non	Rôle système, protégé	

Permission — table accounts_permission

Droit élémentaire (32 au catalogue), identifié par un code. Clé primaire : `id`.

Champ	Type	Description	Contraintes
`created_at`	Date et heure	Date de création (posée par le serveur)	
`updated_at`	Date et heure	Date de dernière modification (posée par le serveur)	
`codename`	Texte (100)	Code de la permission (ex. dossier.validate)	unique
`name`	Texte (200)	Libellé affiché	
`category`	Texte (50)	Famille de permissions	
`description`	Texte long	Description	

Permission d'un rôle — table accounts_rolepermission

Associe un rôle à une permission. Clé primaire : `id`.

Champ	Type	Description	Contraintes
`created_at`	Date et heure	Date de création (posée par le serveur)	
`updated_at`	Date et heure	Date de dernière modification (posée par le serveur)	
`role`	Lien → Role	Rôle concerné	
`permission`	Lien → Permission	Permission concernée	

Rôle d'un utilisateur — table accounts_userrole

Associe un utilisateur à un rôle, avec l'auteur et la date de l'attribution. Clé primaire : `id`.

Champ	Type	Description	Contraintes
`created_at`	Date et heure	Date de création (posée par le serveur)	
`updated_at`	Date et heure	Date de dernière modification (posée par le serveur)	
`user`	Lien → CustomUser	Utilisateur concerné	
`role`	Lien → Role	Rôle concerné	
`is_active`	Oui / non	Attribution en vigueur	
`assigned_at`	Date et heure	Date de l'attribution	
`assigned_by`	Lien → CustomUser	Auteur de l'attribution	facultatif

Ajustement de permission — table accounts_userpermission

Permission accordée ou retirée à un utilisateur en plus (ou à la place) de ses rôles. Clé primaire : `id`.

Champ	Type	Description	Contraintes
`created_at`	Date et heure	Date de création (posée par le serveur)	
`updated_at`	Date et heure	Date de dernière modification (posée par le serveur)	
`user`	Lien → CustomUser	Utilisateur concerné	
`permission`	Lien → Permission	Permission concernée	

Champ	Type	Description	Contraintes
`granted`	Oui / non	Oui : permission accordée ; non : permission retirée	
`set_by`	Lien → CustomUser	Auteur de l'ajustement	facultatif

Établissement de santé — table hospitals_hospital

Hôpital, maternité ou centre de santé qui déclare des naissances. Colonnes communes à toutes les tables métier (non répétées ci-dessous) : `id`, `uuid`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_archived`, `archived_at`, `archived_by`.

Champ	Type	Description	Contraintes
`code`	Texte (20)	Code automatique WH-0001-26 (non modifiable)	unique
`name`	Texte (200)	Nom de l'établissement	
`facility_type`	Texte (20)	Type (hôpital, maternité, centre de santé…)	
`region`	Texte (100)	Région	
`commune`	Texte (100)	Commune	
`address`	Texte long	Adresse / domicile	
`contact_name`	Texte (150)	Personne à contacter	
`phone`	Texte (20)	Téléphone, enregistré au format +224 624 62 94 77	
`email`	E-mail	Adresse e-mail	
`is_active`	Oui / non	Établissement actif ou désactivé	

Compteur de références — table declarations_referencecounter

Compteur verrouillé qui produit les références WKR-AAAA-NNNN et les codes d'établissement. Clé primaire : `id`.

Champ	Type	Description	Contraintes
`key`	Texte (60)	Nature et année du compteur	unique
`value`	Entier	Dernière valeur attribuée	

Déclaration de naissance — table declarations_declaration

Dossier central : statut, établissement, dates de réception, transmission et validation. Colonnes communes à toutes les tables métier (non répétées ci-dessous) : `id`, `uuid`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_archived`, `archived_at`, `archived_by`.

Champ	Type	Description	Contraintes
`reference`	Texte (50)	Référence de suivi interne WKR-AAAA-NNNN (unique)	unique
`status`	Texte (30)	Statut dans le circuit (voir chapitre Workflow)	
`hospital`	Lien → Hospital	Établissement déclarant (peut être vide)	facultatif

Champ	Type	Description	Contraintes
`hospital_document_ref`	Texte (100)	Référence du document remis par l'hôpital	
`doctor_name`	Texte (150)	Médecin	
`midwife_name`	Texte (150)	Sage-femme	
`birth_date`	Date	Date de naissance de l'enfant	
`birth_time`	Heure	Heure de naissance	facultatif
`birth_place`	Texte (200)	Lieu de naissance	
`notes`	Texte long	Observations	
`received_at`	Date	Date de réception du dossier	
`assigned_to`	Lien → CustomUser	Responsable actuel du dossier	facultatif
`last_decision_reason`	Texte long	Dernier motif de retour ou de rejet	
`transmitted_at`	Date et heure	Date de transmission au secrétariat	facultatif
`validated_at`	Date et heure	Date de validation définitive	facultatif
`validated_by`	Lien → CustomUser	Administrateur qui a validé	facultatif

Enfant — table declarations_child

Enfant concerné par la déclaration (une seule fiche par déclaration). Colonnes communes à toutes les tables métier (non répétées ci-dessous) : `id`, `uuid`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_archived`, `archived_at`, `archived_by`.

Champ	Type	Description	Contraintes
`declaration`	Lien 1-1 → Declaration	Déclaration (une seule fiche enfant)	unique
`first_names`	Texte (150)	Prénoms	
`last_name`	Texte (100)	Nom	
`gender`	Texte (1)	Sexe (M ou F)	
`nationality`	Texte (100)	Nationalité (texte libre ou liste des pays)	
`npi`	Texte (50)	Numéro personnel d'identification (texte libre, 50 caractères)	
`birth_rank`	Entier	Rang de naissance chez la mère (1 = premier enfant)	facultatif

Parent — table declarations_parent

Père ou mère de l'enfant (au plus un de chaque type par déclaration). Colonnes communes à toutes les tables métier (non répétées ci-dessous) : `id`, `uuid`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_archived`, `archived_at`, `archived_by`.

Champ	Type	Description	Contraintes
`declaration`	Lien → Declaration	Déclaration concernée	

Champ	Type	Description	Contraintes
`parent_type`	Texte (10)	FATHER (père) ou MOTHER (mère)	
`first_names`	Texte (150)	Prénoms	
`last_name`	Texte (100)	Nom	
`date_of_birth`	Date	Date de naissance	facultatif
`birth_place`	Texte (200)	Lieu de naissance	
`age`	Entier	Âge (calculé à la saisie, modifiable)	facultatif
`nationality`	Texte (100)	Nationalité (texte libre ou liste des pays)	
`npi`	Texte (50)	Numéro personnel d'identification (texte libre, 50 caractères)	
`profession`	Texte (100)	Profession	
`address`	Texte long	Adresse / domicile	
`phone`	Texte (20)	Téléphone, enregistré au format +224 624 62 94 77	
`id_type`	Texte (50)	Type de pièce d'identité	
`id_number`	Texte (50)	Numéro de la pièce d'identité	

Déclarant — table declarations_declarant

Personne qui déclare la naissance (père, mère ou autre). Colonnes communes à toutes les tables métier (non répétées ci-dessous) : `id`, `uuid`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_archived`, `archived_at`, `archived_by`.

Champ	Type	Description	Contraintes
`declaration`	Lien 1-1 → Declaration	Déclaration (un seul déclarant)	unique
`first_names`	Texte (150)	Prénoms	
`last_name`	Texte (100)	Nom	
`relationship`	Texte (50)	Lien avec l'enfant (père, mère, autre)	
`nationality`	Texte (100)	Nationalité (texte libre ou liste des pays)	
`npi`	Texte (50)	Numéro personnel d'identification (texte libre, 50 caractères)	
`profession`	Texte (100)	Profession	
`phone`	Texte (20)	Téléphone, enregistré au format +224 624 62 94 77	
`address`	Texte long	Adresse / domicile	
`id_type`	Texte (50)	Type de pièce d'identité	
`id_number`	Texte (50)	Numéro de la pièce d'identité	

Historique des statuts — table declarations_statushistory

Chaque changement de statut d'un dossier (ajout seul). Clé primaire : `id`.

Champ	Type	Description	Contraintes
`declaration`	Lien → Declaration	Déclaration concernée	
`from_status`	Texte (30)	Statut de départ	
`to_status`	Texte (30)	Nouveau statut	
`changed_by`	Lien → CustomUser	Auteur du changement	facultatif
`changed_at`	Date et heure	Date et heure du changement	
`reason`	Texte long	Motif (obligatoire pour retours et rejets)	

Acte de naissance — table actes_acte

Références du registre, numéro officiel, circonscription et mentions (un acte par déclaration). Colonnes communes à toutes les tables métier (non répétées ci-dessous) : `id`, `uuid`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_archived`, `archived_at`, `archived_by`.

Champ	Type	Description	Contraintes
`declaration`	Lien 1-1 → Declaration	Déclaration (un seul acte)	unique
`annee`	Entier	Année du registre	facultatif
`registre`	Texte (50)	Registre	
`feuillet`	Texte (50)	Feuillet	
`numero_acte`	Texte (50)	Numéro officiel de l'acte (unique par année et registre)	facultatif
`numero_assigned_at`	Date et heure	Date d'attribution du numéro	facultatif
`numero_assigned_by`	Lien → CustomUser	Administrateur qui a attribué le numéro	facultatif
`region`	Texte (100)	Région administrative	
`prefecture`	Texte (100)	Préfecture	
`commune`	Texte (100)	Commune	
`centre`	Texte (150)	Centre d'état civil	
`declaration_date`	Date	Date de la déclaration	facultatif
`officer_name`	Texte (150)	Officier d'état civil (mention)	
`mentions`	Texte long	Mentions complémentaires	

Séquence de numéros — table actes_numbersequence

Dernier numéro attribué par année et registre (numérotation automatique, si activée). Clé primaire : `id`.

Champ	Type	Description	Contraintes
`annee`	Entier	Année	
`registre`	Texte (50)	Registre	
`last_value`	Entier	Dernier numéro attribué	

Impression — table actes_printlog

Chaque impression d'un acte, avec son numéro de copie. Clé primaire : `id`.

Champ	Type	Description	Contraintes
`acte`	Lien → Acte	Acte imprimé	
`printed_by`	Lien → CustomUser	Auteur de l'impression	facultatif
`printed_at`	Date et heure	Date et heure de l'impression	
`copy_number`	Entier	Numéro de la copie	

Délivrance — table actes_delivery

Remise d'un acte à un bénéficiaire ; duplicata si ce n'est pas la première. Colonnes communes à toutes les tables métier (non répétées ci-dessous) : `id`, `uuid`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_archived`, `archived_at`, `archived_by`.

Champ	Type	Description	Contraintes
`acte`	Lien → Acte	Acte délivré	
`delivered_at`	Date et heure	Date et heure de la remise	
`recipient_name`	Texte (200)	Bénéficiaire (remis à)	
`recipient_relationship`	Texte (100)	Qualité du bénéficiaire	
`recipient_id_type`	Texte (50)	Type de pièce du bénéficiaire	
`recipient_id_number`	Texte (50)	Numéro de la pièce	
`is_duplicate`	Oui / non	Oui : duplicata (déjà délivré auparavant)	
`notes`	Texte long	Observations	
`delivered_by`	Lien → CustomUser	Agent qui a délivré	facultatif

Fichier stocké — table documents_storedfile

Contenu binaire d'une pièce lorsque les pièces sont conservées dans la base. Clé primaire : `id`.

Champ	Type	Description	Contraintes
`name`	Texte (255)	Nom aléatoire du fichier (non devinable)	unique
`content`	Binaire	Contenu du fichier	
`size`	Entier	Taille en octets	

Pièce justificative — table documents_supportingdocument

Photo ou fichier joint à un dossier (fiche physique, pièce d'identité). Colonnes communes à toutes les tables métier (non répétées ci-dessous) : `id`, `uuid`, `created_at`, `updated_at`, `created_by`, `updated_by`, `is_archived`, `archived_at`, `archived_by`.

Champ	Type	Description	Contraintes
`declaration`	Lien → Declaration	Déclaration concernée	
`kind`	Texte (20)	Nature (PAPER_FORM : fiche physique, ou autre pièce)	
`file`	Fichier	Emplacement du fichier (privé)	
`original_name`	Texte (255)	Nom d'origine du fichier	
`content_type`	Texte (100)	Type de fichier contrôlé	
`size`	Entier	Taille en octets	
`sha256`	Texte (64)	Empreinte SHA-256 du contenu	
`description`	Texte (255)	Description	
`uploaded_by`	Lien → CustomUser	Auteur de l'ajout	facultatif

Journal d'audit — table audit_auditlog

Trace de chaque opération : auteur, action, objet, modifications, motif, adresse IP (ajout seul). Clé primaire : `id`.

Champ	Type	Description	Contraintes
`created_at`	Date et heure	Date et heure de l'opération	
`user`	Lien → CustomUser	Auteur (lien)	facultatif
`username`	Texte (150)	Identifiant de l'auteur, conservé tel quel	
`action`	Texte (40)	Type d'action (connexion, création, modification, validation…)	
`object_type`	Texte (60)	Type d'objet concerné	
`object_id`	Texte (64)	Identifiant de l'objet	
`object_repr`	Texte (255)	Description lisible de l'objet	
`changes`	JSON	Modifications champ par champ (ancienne et nouvelle valeur)	
`reason`	Texte long	Motif	
`ip_address`	Adresse IP	Adresse IP du client	facultatif
`user_agent`	Texte (255)	Navigateur utilisé	

Paramètre du site — table core_sitesetting

Paramètre modifiable dans l'application (textes de connexion, valeurs par défaut). Clé primaire : `id`.

Champ	Type	Description	Contraintes
`created_at`	Date et heure	Date de création (posée par le serveur)	
`updated_at`	Date et heure	Date de dernière modification (posée par le serveur)	
`key`	Texte (100)	Clé du paramètre	unique
`value`	Texte long	Valeur	

Champ	Type	Description	Contraintes
`description`	Texte (255)	Description	
`is_public`	Oui / non	Visible sur la page de connexion	
`updated_by`	Lien → CustomUser	Auteur de la modification	facultatif

12.2 Annexe B — Commandes de gestion

Commande	Rôle
`python manage.py migrate`	Applique les migrations de la base
`python manage.py setup_initial_data`	Crée (sans doublon) permissions, rôles, paramètres, comptes de départ et établissements de Ratoma (une seule fois). Option `--no-hospitals` pour ignorer les établissements
`python manage.py seed_hospitals`	Préremplit à la demande les établissements de Ratoma
`python manage.py seed_demo --count 45`	Données de **démonstration fictives** ; refusée si `DEBUG=False`
`python manage.py backup_wakara --output-dir <dossier> --keep 30`	Sauvegarde la base et les pièces dans une archive datée
`python manage.py restore_wakara <archive> --yes`	Restaure une archive (arrêter l'application avant)
`python manage.py check --deploy`	Contrôle des réglages de production
`python manage.py test tests`	Lance les 161 tests automatisés

12.3 Annexe C — Glossaire

Terme	Définition
CECI	Centre d'état civil et de l'identification de Ratoma
Déclaration	Dossier saisi à partir de la fiche reçue de l'hôpital ; une déclaration = un enfant
Acte	Acte de naissance établi à partir de la déclaration, avec son numéro officiel
Référence	Numéro de suivi interne `WKR-AAAA-NNNN`, différent du numéro officiel de l'acte
NPI	Numéro personnel d'identification (texte libre dans la version 1.0)
Duplicata	Délivrance d'un acte déjà délivré auparavant
JWT	Jeton signé qui prouve l'identité de l'utilisateur pendant une durée limitée
Migration	Fichier versionné qui fait évoluer la structure de la base de données
Ajout seul	Table où l'on peut seulement ajouter des lignes, jamais les modifier ni les effacer

12.4 Annexe D — Références

- Dépôt du code : https://github.com/alhassaneplea4/wakara-rtm1

- Documentation du dépôt : dossier `docs/` (installation, architecture, configuration, déploiement, sauvegarde, décisions métier).
- Autres documents du dossier : 01 cahier des charges, 03 manuel utilisateur, 04 guide d'installation et d'exploitation, 05 rapport de tests, 06 PV de recette, 07 PV de livraison.